



LABORATORY 1

Getting started with ROS 2 and the BlueROV2

Explore a simulated ROV, discover ROS topics, and send your first motion command from Python.

Course	INRO - Instrumentation, Marine Robotics and Power Systems
Professor	Daniel Mihai Toma
Duration	One laboratory session; install software before class
Level	No previous Python or ROS 2 experience required

Mission

Find the ROV sensor and thruster topics. Observe a short dive. Then edit one Python number to make the ROV descend more gently.

You will not write a feedback controller or perform the CTD experiment today. Those are introduced in a later laboratory.

1. Learning outcomes

This is the first ROS 2 and Python session. After this laboratory you should be able to:

- launch the BlueROV2 simulation in a repeatable Dev Container;
- find ROS 2 nodes and topics, including the ROV sensor and command topics;
- observe depth and individual thruster forces during a short movement;
- change one value in a provided Python program to send a safe motion command.

Laboratory sequence

Stage	Student action	Evidence
A. Prepare	Install Git, Docker, VS Code and Dev Containers before class.	Container opens.
B. Launch	Build the package and start the simulated ROV.	Depth topic publishes.
C. Explore	List nodes/topics and observe sensors and thrusters.	Completed topic worksheet.
D. Command	Run a short motion and edit one Python effort value.	Two depth observations.
E. Explain	Relate command sign and magnitude to observed motion.	Brief written answer.

Scope

This exercise introduces communication and observation. Laboratory 2 adds subscribers, publishers and CSV trajectory logging. Feedback control and the CTD density experiment are reserved for Laboratory 3.

Files used today

File	Purpose
launch/demo.launch.py	Starts the Gazebo pool, bridge and thruster mixer.
scripts/drive.py	Sends a short instructor-provided motion command.
lab01/first_dive.py	Your first Python edit: one heave-effort value.

2. Install the software

Complete this chapter before the laboratory. Docker runs Ubuntu 24.04, ROS 2 Jazzy, Gazebo Harmonic and Python inside the Dev Container. You do not install those packages directly on your computer.

2.1 Windows: enable or update WSL 2

Open PowerShell as Administrator. If WSL is not installed, run `wsl --install`. Then update and check it:

```
wsl --install
wsl --update
wsl --version
```

Restart Windows if requested. Hardware virtualization must be enabled. Check [Docker's current Windows requirements](#) for supported Windows/WSL versions and memory.

2.2 Windows: install Docker Desktop

- 1 Download [Docker Desktop for Windows](#). The usual per-user installation is appropriate.
- 2 Choose the WSL 2 backend for Linux containers when the installer asks.
- 3 Start Docker Desktop. Wait until it reports that its engine is running.
- 4 In PowerShell, run `docker version`. Both Client and Server should appear.

2.3 Windows: install Git and VS Code

- 1 Install [Git for Windows](#); check with `git --version`.
- 2 Install [Visual Studio Code](#).
- 3 Open VS Code Extensions and install Dev Containers by Microsoft.

Why a Dev Container?

Every student uses the same Linux tools while editing a local copy of the repository. The course Dockerfile defines the environment; the repository supplies the exercises.

2. Install the software (Ubuntu)

2.4 Ubuntu: install Docker Engine

These instructions target a 64-bit installation of Ubuntu 24.04 LTS. Use Docker's official repository so VS Code can build and run the course Dev Container.

```
sudo apt update
sudo apt install ca-certificates curl git x11-xserver-utils
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources > /dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

Allow your normal user to use Docker, then sign out of Ubuntu and sign in again:

```
sudo usermod -aG docker $USER
```

Test the installation without sudo:

```
docker run --rm hello-world
docker version
```

Docker permission

Membership in the docker group grants root-level privileges. Use it only on a computer you administer and run trusted container projects.

2.5 Ubuntu: install VS Code and enable the Gazebo window

- 1 Install [Visual Studio Code for Linux](#) using Microsoft's Debian/Ubuntu instructions.
- 2 Open VS Code Extensions and install Dev Containers by Microsoft.
- 3 Before opening the container, run `xhost +si:localuser:root` in an Ubuntu terminal. This lets the root user inside the course container use your X display.
- 4 Choose INRO - Ubuntu ROS 2 Jazzy + Gazebo Harmonic, stored in `.devcontainer/ubuntu/devcontainer.json`.

If your desktop does not permit the Gazebo window, launch with `gui:=false`. The simulation, ROS topics and Python exercise still run in the terminal.

2. Install the software (Mac)

2.6 macOS: install the host tools

- 1 Install the [Docker Desktop edition for your Mac](#) (Apple silicon or Intel). Start it and wait for the engine.
- 2 Install Git if needed using [Git's macOS instructions](#); check with `git --version`.
- 3 Install [VS Code](#) and Microsoft's Dev Containers extension.
- 4 In Terminal, run `docker version`. Both Client and Server should appear.

2.7 Use the Mac headless profile

Choose the repository configuration named INRO - Mac headless ROS 2 Jazzy + Gazebo Harmonic. Its file is `.devcontainer/mac/devcontainer.json`. It uses the same Dockerfile but omits the Windows-only WSLg display mount.

What works on Mac?

The intended Mac route runs Gazebo physics without a 3D window. Students still use the same ROS topics, depth values, thruster forces and Python command. Use `gui:=false` when launching.

The ROS base image provides both Intel and ARM64 variants. Gazebo Harmonic lists ARM platforms as best-effort, and this course profile has not been tested on a physical Mac. In particular, a Gazebo 3D window through Docker Desktop is not part of the supported Mac instructions. If the image build fails on an Apple silicon Mac, use a validated laboratory Windows PC or contact the instructor.

2.8 Access to the course repository

The GitHub repository is private. Ask the instructor to grant your GitHub account access before class. Your GitHub login may appear in a browser during cloning.

3. Clone, open and launch

3.1 Clone the repository once

Windows PowerShell: choose a normal local folder such as Documents, outside system folders and cloud-synced virtual drives.

```
cd $HOME\Documents
mkdir GitHub
cd GitHub
git clone https://github.com/danielMihaiToma/INRO-Marine-Robotics.git
cd INRO-Marine-Robotics
code .
```

Ubuntu or Mac Terminal: use the same GitHub URL. Then open the cloned folder with VS Code File → Open Folder.

```
mkdir -p ~/Documents/GitHub
cd ~/Documents/GitHub
git clone https://github.com/danielMihaiToma/INRO-Marine-Robotics.git
```

3.2 Open the Dev Container

- 1 Start Docker Desktop on Windows or Mac. On Ubuntu, check that docker version works. Then open the INRO-Marine-Robotics folder in VS Code.
- 2 Open the Command Palette: Ctrl+Shift+P on Windows/Ubuntu or Cmd+Shift+P on Mac. Run Dev Containers: Reopen in Container.
- 3 Select the normal Windows configuration, Ubuntu configuration, or Mac headless configuration for your computer. If VS Code opens the wrong profile, use Dev Containers: Open Folder in Container... and choose the correct configuration.
- 4 Wait for the first image build. Open a new VS Code terminal; its path should begin with /workspaces/.

Already inside?

If VS Code offers Reopen Folder Locally, you are already connected to a container. Check its name in the lower-left status area.

3.3 Build and launch the ROV

```
cd /workspaces/INRO-Marine-Robotics/ros2_ws
python3 src/inro_rov_demo/scripts/prepare_model.py
colcon build --packages-select inro_rov_demo --symlink-install
source install/setup.bash
```

Launch with a window on Windows or Ubuntu: `ros2 launch inro_rov_demo demo.launch.py`. On a Mac, run `ros2 launch inro_rov_demo demo.launch.py gui:=false`. Keep this terminal open. In each additional terminal, run `source install/setup.bash`.

Gazebo camera: right-click bluerov2, choose Move to, zoom with the mouse wheel, then choose Follow. Follow does not zoom by itself.

4. Discover ROS nodes and topics

A ROS node is a running program. A topic is a named stream of messages. Gazebo, the bridge and the thruster mixer communicate through these streams.

The idea

Gazebo ROV → ROS bridge → sensor topics → your terminal

Your Python command → /inro/command → thruster mixer → six thruster-force topics → Gazebo ROV

Open a second terminal inside the same Dev Container

```
cd /workspaces/INRO-Marine-Robotics/ros2_ws
source install/setup.bash
ros2 node list
ros2 topic list -t
ros2 topic info /inro/command
```

Use the output to complete this worksheet. Topic names are case-sensitive.

Find a topic that...	Topic name	Message type
reports depth		
reports position/velocity		
reports IMU readings		
receives a vehicle command		
sends force to one thruster		

The six thruster topics are /inro/thruster_1/force through /inro/thruster_6/force. There is no single /inro/thrusters topic.

Questions

How many /inro/ topics can you find? Which node appears to convert one vehicle command into individual thruster commands?

5. Observe a short movement

In the second terminal, display the ROV depth. Stop the stream with Ctrl+C when you have enough data.

```
ros2 topic echo /inro/depth
```

Open a third terminal, run `source install/setup.bash`, and send a timed downward command:

```
ros2 run inro_rov_demo drive down --seconds 4 --power 0.4
```

Depth is measured in metres and increases downward. Restart the simulator between trials if you want to compare from the same starting pose.

Explore sensors and thrusters

Use `ros2 topic echo TOPIC_NAME` to inspect `/inro/odometry`, `/inro/imu`, and `/inro/thruster_5/force` or `/inro/thruster_6/force`. You do not need to understand every field yet. Then try:

```
ros2 run inro_rov_demo drive forward --seconds 3 --power 0.4
ros2 run inro_rov_demo drive left --seconds 3 --power 0.4
```

Run these separately. Observe which thruster forces change and what the ROV does.

Motion	Thrusters that change	What did the ROV do?
Down		
Forward		
Left turn		
Command finished		

Important observation

A zero command stops thrust; it does not instantly stop motion or hold depth. Inertia and buoyancy can still move the ROV. An automatic depth hold requires feedback control in Laboratory 3.

6. Your first Python command

The course provides the working ROS 2 program. You only edit one number.

```
ros2_ws/src/inro_rov_demo/lab01/first_dive.py
```

Find this line near the top:

```
HEAVE_EFFORT = -0.40
```

The program repeatedly publishes [surge, heave, yaw] to /inro/command for four seconds. Here only the middle value changes. Negative heave requests downward thrust; positive heave requests upward thrust.

Value	Prediction before running	Observation after running
-0.40		
-0.20		
+0.20 (optional)		

Run the script

Keep demo.launch.py running. In another sourced container terminal:

```
cd /workspaces/INRO-Marine-Robotics/ros2_ws
source install/setup.bash
python3 src/inro_rov_demo/lab01/first_dive.py
```

- 1 Predict the motion with -0.40. Run the script and record depth before and after.
- 2 Restart the simulator. Change the one line to -0.20 and repeat.
- 3 Optional: predict and try +0.20. Keep the effort magnitude at or below 0.65.

Effort is not speed

The values -0.40 and -0.20 are normalized thrust requests, not speeds in m/s. The achieved speed depends on drag, inertia and buoyancy. Feedback speed control is introduced in Laboratory 3.

7. Submit and look ahead

What to submit

- Your edited `first_dive.py`.
- The ROS topic worksheet from page 7.
- The motion-and-thruster table from page 8.
- Depth before and after two Python trials, plus two or three sentences explaining the sign and magnitude of heave effort.

Troubleshooting

Problem	Check
No "Reopen in Container" option	Is Docker Desktop running? Is the Microsoft Dev Containers extension installed? If VS Code offers "Reopen Folder Locally," you are already inside.
ROS package not found	Build from <code>ros2_ws</code> , then source <code>install/setup.bash</code> in each new container terminal.
No Gazebo window	Mac headless mode intentionally has no window. On Windows, check WSLg. On Ubuntu, run <code>xhost +si:localuser:root</code> before reopening the container, or use <code>gui:=false</code> .
Python reports no mixer	Start <code>demo.launch.py</code> first, and run only one copy of the simulator.
ROV becomes hard to see	Right-click <code>bluerov2</code> → Move to, zoom with the wheel, then Follow.
Depth changes after command ends	This is expected: zero thrust is not depth hold.

Next laboratory

Laboratory 2: ROS 2 data and motion

You will write a subscriber that records odometry and depth to CSV, then a publisher that commands a simple descent, forward motion and ascent. Laboratory 3 introduces feedback depth control and the CTD density profile.

References

- [Docker Desktop installation for Windows](#)
- [Docker Engine installation for Ubuntu](#)
- [Docker Linux post-installation steps](#)
- [Docker Desktop installation for Mac](#)
- [VS Code Dev Containers guide](#)
- [GitHub cloning guide](#)
- [ROS 2 Jazzy tutorials](#)

Use these commands only with the simulation, never with a real ROV.